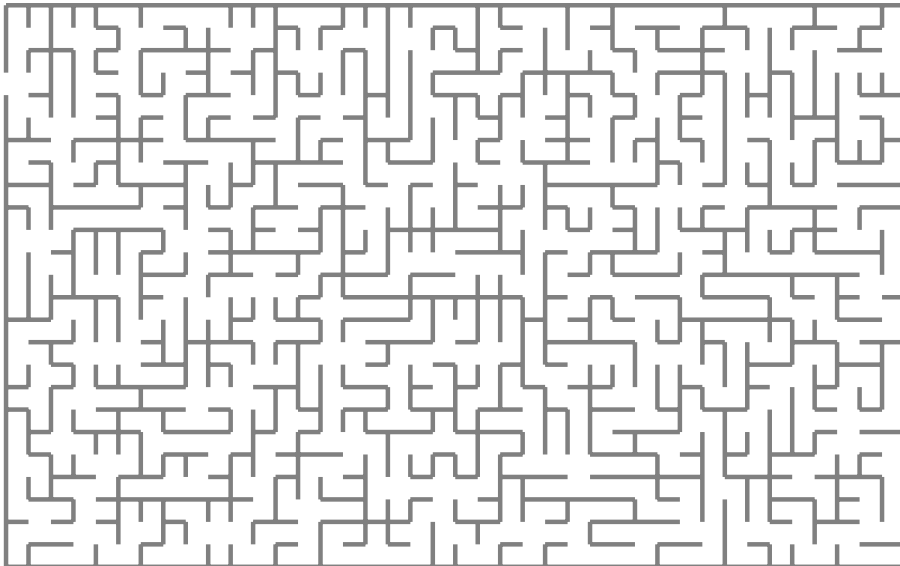


Software Engineering Master Thesis Template

Version of 30th April 2026



The Author

Software Engineering Master Thesis Template

THESIS

submitted in partial fulfilment of the
requirements for the degree of

MASTER OF SCIENCE

in

SOFTWARE ENGINEERING

by

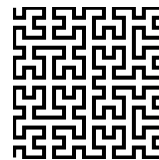
The Author

under the supervision of
Dr. Ana Maria Oprescu (CCI, UvA)

April 2026



Master Software Engineering
Informatics Institute
FNWI, University of Amsterdam
Amsterdam, The Netherlands



Hilbert Institute for Space Research
Strekkerweg 41
1033 DA Amsterdam
The Netherlands

Thesis Committee

Examiner (chair):	Dr. Ana Maria Oprescu (CCI, UvA)
Second Reviewer:	Dr. Thomas L. van Binsbergen (CCI, UvA)
Daily Supervisor:	Dr. Damian Frölich (CCI, UvA)
External Supervisor:	Dennis Bijlsma (SIG)
Other members:	Prof. dr. ir. Cees Bakker (VU), Dr. Pierre de Vries (RUG)

Copyright © The Author, 2026. *Note that this notice is for demonstration purposes and that the $\text{\LaTeX}$ style and document source are free to use as basis for your MSc thesis.*

Cover picture: A “random” maze.

Abstract

This is your abstract, where you succinctly describe the context, the problem, your proposed solution and the conclusions. It should be at least 200 words, but no more than a page, and avoid any citations.

Contents

Abstract	v
Contents	vii
List of Figures	ix
List of Tables	x
Acknowledgements	xi
1 Introduction	1
1.1 Context	1
1.2 Problem Statement	1
1.3 Research Questions	1
1.4 Approach	2
1.5 Outline	2
2 Background	3
3 Related Work	5
4 Methods	7
5 Experiments & Results	9
6 Analysis	11
7 Conclusion	13
7.1 Summary & Findings	13
7.2 Contributions	13
7.3 Limitations & Threats to Validity	13
7.4 Future Work	13

A	Appendix	15
B	Showcase	17
B.1	References	17
B.2	Tables	18
B.3	Cleveref	18
B.4	Glossary	18
B.5	Todonotes	19
B.6	Syntax Highlighting	19
	References	21
	Acronyms	23

List of Figures

B.1	Structure of a trebuchet.	19
-----	-----------------------------------	----

List of Tables

B.1	Booktabs example table	18
-----	----------------------------------	----

Acknowledgements

This is were you can acknowledged people, if you prefer to do so.

Introduction

This is the template to use for your master thesis Software Engineering. It provides you with a structure that we expect you to follow, with guidelines on the contents of each section. Depending on your research you can deviate from this structure, usually by renaming or adding some chapters in the middle. Do discuss this with your academic supervisor first! Be sure to change the variables under the `Options` banner set in `main.tex`, such as title, subtitle, author, supervisor, company, *etc.*.

The Introduction chapter itself is to provide your reader with the context of your research, the problem you are trying to solve.

1.1 Context

Provide the context of your research, and explain the importance and relevance of your topic within the field of software engineering.

1.2 Problem Statement

Give a concise description of the challenge you aim to address. The goal is to define the problem in clear terms, establish its significance, and provides a foundation for developing objectives, methods, and potential solutions. Be specific and avoid vague language.

1.3 Research Questions

Explicitly list the research questions or hypotheses you address in your research. The research questions should directly address the problem statement, as defined in the previous section, and guide your research methodology, as described in the next section. It is helpful to enumerate them, for example as *RQ1*, *RQ2*, so you can refer to them in later chapters.

1.4 Approach

Give a brief overview of the approach you will take to answer the research questions. Define the research methods you used (*e.g.* case study, systematic literature review, action research, *etc.*), and the general steps you will take to conduct your research. This will provide the reader with a high-level roadmap.

1.5 Outline

Give the reader a short overview of the structure of your thesis, and what to find in each chapter.

Background

In this chapter, you provide information that has a benefit for an informed audience. Often, you use this to explain a technique or a concept you use, referring to the literature. This is different from Related Work (chapter 3), where you show the relation of literature to your contributions (to ground, compare, or inform). A good rule of thumb is that your thesis should still make sense if the background chapter is removed.

Related Work

In this chapter, you describe how other people have dealt with similar problems that you are dealing with. What were their techniques? What were their results? What observations did they make that you have used? In what way is your work different?

The role of the related work is threefold:

1. Your work is grounded in earlier work done by other researchers. That means you use techniques and methods, ways of measuring and evaluating, datasets and terminology used by others working on the same or a related problem before you.
2. You want to compare your results with those of others. This is especially true for key papers with respect to your research, *i.e.* State-Of-The-Art (SOTA). Those papers need to be referred to in this chapter. This could also lead to a hypothesis, which is the answer that you expect based on previous literature.
3. The previous two points mentioned cover the main function of this chapter. In addition with related work, you inform the reader on the SOTA, the state of the problem, and the existing solutions. You also show that you know what you are talking about, that you know what is “for sale”, and that you use the best there is.

Depending on your story, you may want to put this chapter at the end of your thesis (before the Conclusions chapter) or at the beginning of your thesis (after the Introduction chapter). Typically, when your work is a direct extension or modification of existing work, you want to put this at the beginning to compare and contrast your research. This helps in showing the reader the context and demonstrating the novelty of your approach.

When your research is in a very new area where you explore a novel concept, you may want to put the Related Work at the end. It is beneficial to first get the reader comfortable with your research before showing how it fits within the wider research area. Showing the related work first may distract the reader and make them lose interest in the rest of the thesis.

In the end, think about the story you want to tell and discuss this with your academic supervisor.

Methods

In this chapter, you described the design of your research: which methods and procedures you used as a step towards answering the research questions. A well-written methods chapter highlights the plausibility of your research methods. You should clarify to the reader *why* you have chosen specific methods and how they are justified for your research. To garner more credibility, you can include the pitfalls and difficulties associated with your approach.

Depending on the nature of your research, you may, for example, want to describe the exact procedure you use to collect the data, how it is analysed, and how you plan to validate the data and results. On the other hand, when you develop a new framework, you want to describe in detail the requirements and the design, and the motivation behind them (*i.e.* the decision you made and why). Make sure to discuss all ethical aspects of your research. This is more important when dealing with (datasets of) people or (trained) AI models.

Experiments & Results

If your method is to do an experiment, in this chapter you explain how you set up your experiments, what parameters you consider, and the results you got. You can also call this chapter “Design” or “Implementation”, if you are designing a framework or building a prototype. Describe the architecture and design of the software systems you developed. Provide details about the implementation process, and the technologies and tools you have used. Describe the testing process and results, to show the validity and performance of the system. You can then use these results in the next chapter to prove that your system meets the requirements presented in the previous chapter. Be sure to provide a detailed description of the environment, tools, and procedures used in your experiments. This is crucial for reproducibility.

Analysis

With the design and experiments done, and the results presented, now comes the most important part of your research: analysis. In this chapter you discuss the implications of your results, and how they answer your research questions. Be sure to cover in your analysis:

- *Validation*: how do you know that the results you got (and the conclusions you drawing from them) are true,
- *Evaluation*: how well is your system performing.

Conclusion

The Conclusion chapter is typically the last chapter you write, though most often it will be the first chapter others will read (after the abstract). Make sure to be concise and complete, and make it not too detailed. No new information is presented or derived, all information should have been presented in other chapters.

7.1 Summary & Findings

Here you summarise your work (the context, questions, approach), and present the findings in context with the research questions.

7.2 Contributions

Highlight the contributions your thesis provides. These are tangible results, which can live outside the context of this thesis. Examples are new methods, frameworks, software. Make sure to provide links to Git repos or datasets if applicable.

7.3 Limitations & Threats to Validity

No work is perfect. Discuss the limitations of your study. A research study can only be as unbiased as the researcher and the circumstances they are working with. The threats to validity offers insights into the variables that were encountered and how you compensated for them.

7.4 Future Work

This is the moment to highlight all the work you wanted to do if you had more time. Or any new ideas you did not have the time to explore. This is also where you can give an idea how to address any limitations mentioned in the previous section.

Appendix

In the appendix you can provide non-crucial information. Often you put here tables or figures that are too big and do not contribute to the results or evaluation chapter. Same with longer code examples, or design files.

Showcase

This is a showcase of the various packages and features that you can use with this template. You are more than welcome to load your own packages for features you would like to use.

For each package, we also provide a (clickable) link to the Comprehensive T_EX Archive Network (CTAN), as `ctan:package-name`.

B.1 References

For keeping track of your references, you can use the `ctan:biblatex` package. References are stored in `references.bib` in the root of the project. To cite a paper, use `\cite{key}`, for example `\cite{ISO25010}`, becomes [3]. When using a references in a sentence, you can use `\textcite`, for example:

```
\textcite{Beck2000a} argue that programmers should love writing tests.
```

becomes

Beck and Gamma [2] argue that programmers should love writing tests.

You can also directly cite the author by using `\citeauthor{key}` and `\citetitle{key}` for the title. For example:

```
As \citeauthor{Beck2000b} shows in \citetitle{Beck2000b}~\cite{Beck2000b}
```

becomes

As Beck shows in *Extreme programming eXplained: embrace change* [1]

When using Overleaf, you can also link your Zotero account¹. You can choose to either synchronise your entire Zotero library with the `references.bib`, or use the “advanced reference search” to add an entry to `references.bib` whenever you use a reference in your writing.

¹See “How to link Zotero to your Overleaf account”

Item		
Animal	Description	Price (\$)
Gnat	per gram	13.65
	each	0.01
Gnu	stuffed	92.50
Emu	stuffed	33.33
Armadillo	frozen	8.99

Table B.1: Booktabs example table

B.2 Tables

For making publication quality tables, you can use the `ctan:booktabs` package. An example is shown in table B.1. This package allows you to make clearer and visually pleasing tables, by providing additional space above and below rules, and rules of varying ‘thickness’. In this design, vertical and double rules are also left out as it often causes a distraction.

B.3 Cleveref

To save the effort of having to type, *e.g.* “`.. see Table~\ref{tab:a-table}..`”, every time you use a cross-reference, you can make use of `ctan:cleveref`. To cross-reference tables, figures, listings, *etc.*, you can use `\cref{tab:a-table}`, which will automatically provide the right format. Use `\Cref{..}` for the capital version at the beginning of a sentence.

B.4 Glossary

You can make a glossary using `ctan:glossaries` for all the acronyms you use. We can define them in a separate `acronyms.tex`.

In each line, the format is `\newacronym{label}{short}{long}`, whereby `label` is the label you can use to refer to the acronym (can be the same as the actual acronym), `short` is the actual acronym and `long` is the expanded version.

In your text, whenever you want to use an acronym (for example ‘CTAN’), you can refer to it using `\ac{CTAN}`. On first use, the acronym will be expanded, with the acronym in parentheses (as shown in the introduction of this chapter with the acronym CTAN). Subsequent uses will display the acronym in its short form. All acronyms will also link to the glossary, which you can print using `\printacronyms[style=long]` (also see `main.tex`).

You can create a glossary using the command `\makeglossaries{}`. This will generate a glossary of all the acronyms you have used in your text. If you want to include all the acronyms you have defined (but not necessarily used in your text), you can use the starred version of the command (*i.e.* `\makeglossaries*`).

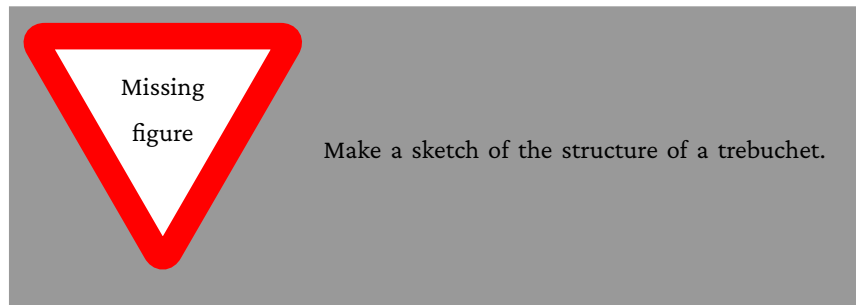


Figure B.1: Structure of a trebuchet.

B.5 Tdonotes

The package `ctan:todonotes` allows you to insert todo items in your document. The simplest form is by simply using `\todo{note}` to insert a note in the margin. You can also use inline notes for things that require more attention,

like a note to say you need to expand this section.

like this for example.

What is also quite useful is that you can create placeholder images with todo notes, as shown in fig. B.1. With `\listoftodos` you can generate a list of todos as shown at the beginning of this template in `main.tex`.

Alternatively, you can use the built-in review functionality of Overleaf.

B.6 Syntax Highlighting

For pretty printing of source code, you can use `ctan:minted`. You can directly show source code via the `minted` environment, as shown in listing 1, or import source code via `\inputminted`, as shown in listing 2.

```
1  #include <stdio.h>
2  int main() {
3      int year;
4      printf("Enter a year: ");
5      scanf("%d", &year);
6
7      // leap year if perfectly divisible by 400
8      if (year % 400 == 0) {
9          printf("%d is a leap year.", year);
10     }
11     // not a leap year if divisible by 100
12     // but not divisible by 400
13     else if (year % 100 == 0) {
14         printf("%d is not a leap year.", year);
15     }
16     // leap year if not divisible by 100
17     // but divisible by 4
18     else if (year % 4 == 0) {
19         printf("%d is a leap year.", year);
20     }
21     // all other years are not leap years
22     else {
23         printf("%d is not a leap year.", year);
24     }
25
26     return 0;
27 }
```

Listing 1: Calculate leap years in C.

```
1  defer bubble-test
2  ' > is bubble-test
3
4  : bubble { addr cnt -- }
5      cnt 1 do
6          addr cnt i - cells bounds do
7              i 2@ bubble-test if i 2@ swap i 2! then
8              cell +loop
9      loop ;
```

Listing 2: Bubble sort in Forth.

References

- [1] K. Beck. *Extreme programming eXplained: embrace change*. Addison-Wesley, 2000. isbn: 0201616416.
- [2] K. Beck and E. Gamma. “Test-infected: programmers love writing tests”. In: *More Java Gems*. Ed. by D. Deugo. SIGS Reference Library. Cambridge University Press, 2000, pp. 357–376. doi: 10.1017/CBO9780511550881.029.
- [3] ISO Central Secretary. *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. en. Standard ISO/IEC TR 29110-1:2016. International Organization for Standardization, 2011. url: <https://www.iso.org/standard/78176.html>.

Acronyms

CTAN Comprehensive TeX Archive Network

SOTA State-Of-The-Art

